

Final Report: ArtConnect Application

Step 1 : Understanding ArtConnect and Defining the Functional Scope

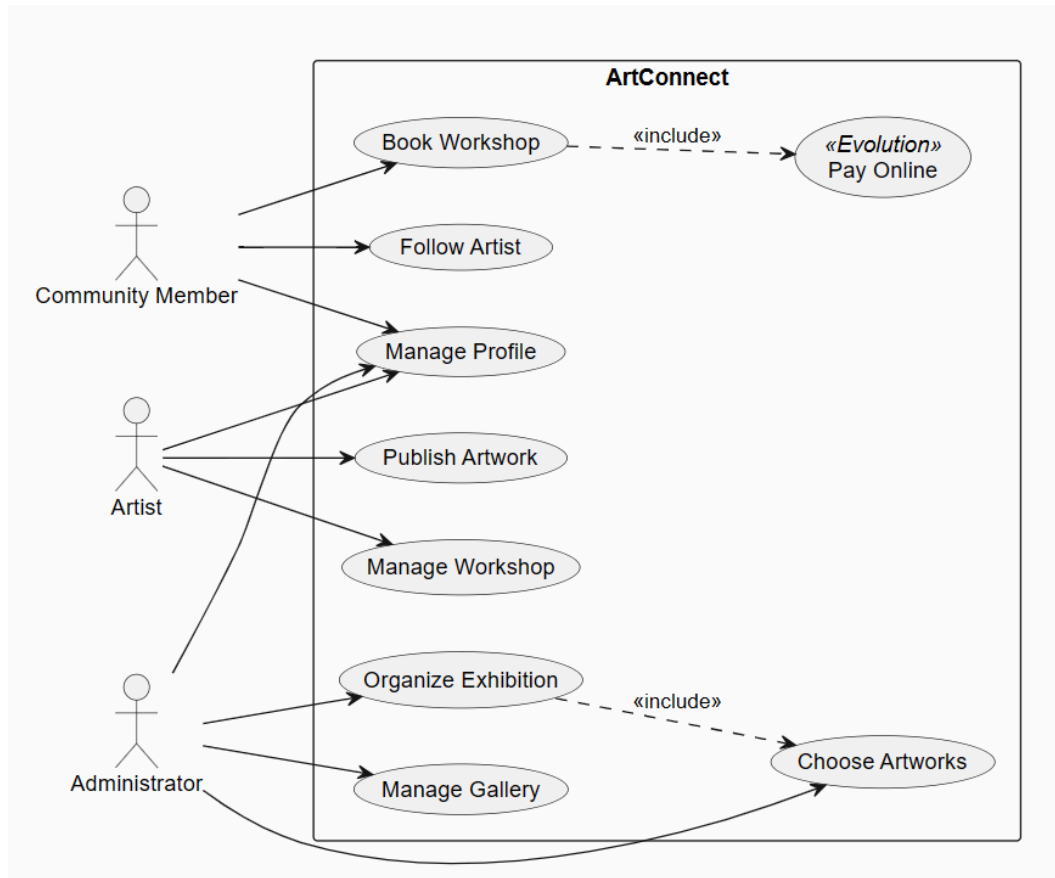
A. User Roles and Profiles

- **Community Member:** The visitor and consumer of art. They explore the platform, follow their favorite artists, book spots in creative workshops, and leave reviews on artworks.
- **Artist:** The creator and instructor. They manage their portfolio by publishing artworks and organize/teach workshops for the community.
- **Administrator:** The platform manager. They oversee the galleries and are responsible for organizing exhibitions and curating the artworks displayed in them.

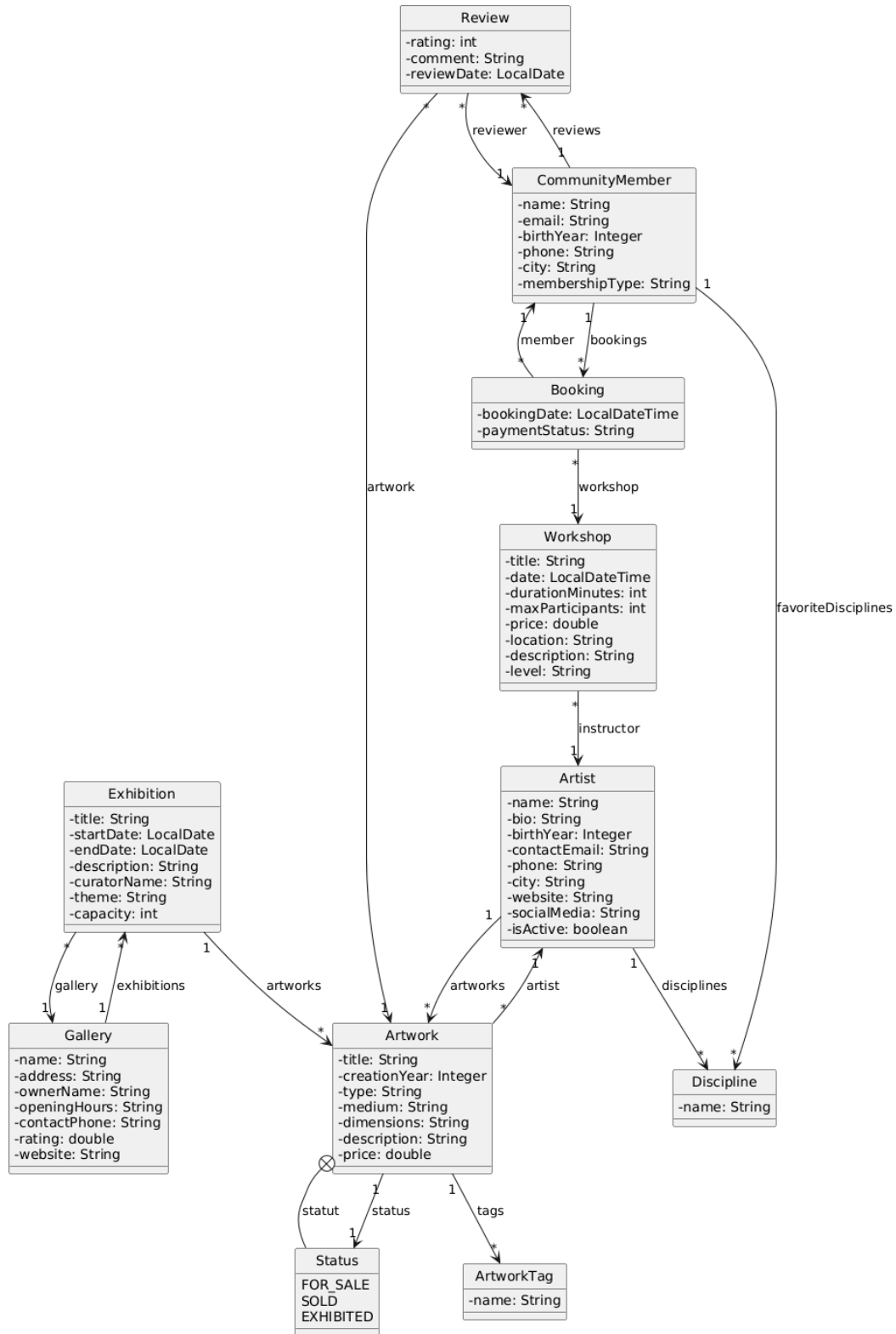
B. Main Features

- **Account & Profile Management:** Registration and profile handling tailored to the specific user type (Member, Artist, or Admin).
- **Social & Discovery:** Ability for community members to follow artists and write reviews for specific artworks.
- **Artwork Management:** Artists can add, categorize, and tag artworks, as well as track their status (for_sale, sold, exhibited).
- **Event Management (Exhibitions):** Administrators can manage physical galleries, schedule exhibitions, and select specific artworks to be displayed.
- **Event Management (Workshops):** Artists can create educational workshops (defining capacity, price, and level), while members can book these workshops, generating a booking record with a payment status.

Use case Diagram :



Class Diagram :



Step 2 : Conceptual and Logical Modeling of the ArtConnect Database

1. Identification of Entities and Relationships

UserAccount: user_id, email, name, password, role

CommunityMember: member_id, join_date, city, membership_type, user_id

Artist: artist_id, biography, discipline, birth_year, city, user_id

Administrator: admin_id, admin_level, user_id

Artwork: artwork_id, title, description, type, creation_date, artist_id

Gallery: gallery_id, address, name, rating, admin_id

Exhibition: exhibition_id, title, description, event_date, capacity, gallery_id

Workshop: workshop_id, title, description, workshop_date, location, capacity, price, level, artist_id

WorkshopBooking: booking_id, booking_date, payment_status, member_id, workshop_id

Follows: member_id, artist_id

presented_in: artwork_id, exhibition_id

2. Functional Dependencies

The functional dependencies define how attributes depend on identifiers:

UserAccount: user_id → email, name, password, role

CommunityMember: member_id → join_date, city, membership_type, user_id

Artist: artist_id → biography, discipline, birth_year, city, user_id

Administrator: admin_id → admin_level, user_id

Artwork: artwork_id → title, description, type, creation_date, artist_id

Gallery: gallery_id → address, name, rating, admin_id

Exhibition: exhibition_id → title, description, event_date, capacity, gallery_id

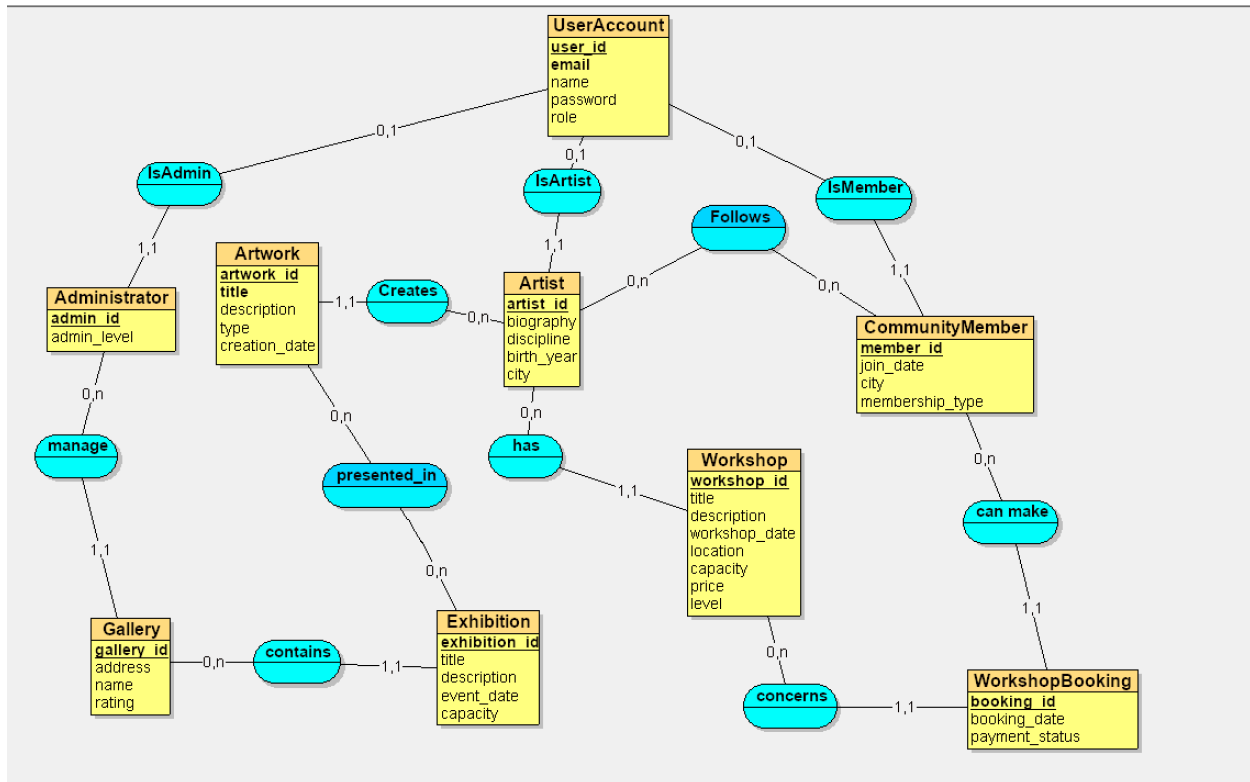
Workshop: workshop_id → title, description, workshop_date, location, capacity, price, level, artist_id

WorkshopBooking: booking_id → booking_date, payment_status, member_id, workshop_id

Follows: (member_id, artist_id) → member follows artist

presented_in: (artwork_id, exhibition_id) → artwork is presented in exhibition

3. Conceptual Data Model



4. Logical Data Model

UserAccount = (user_id , email, name, password , role);

CommunityMember = (member_id, join_date , city , membership_type , #user_id);

Artist = (artist_id, biography , discipline, birth_year , city , #user_id);

Administrator = (admin_id, admin_level, #user_id);

Artwork = (artwork_id, title, description , type, creation_date , #artist_id);

Gallery = (gallery_id, address , name , rating , #admin_id);

Exhibition = (exhibition_id, title , description , event_date , capacity , #gallery_id);

Workshop = (workshop_id, title , description, workshop_date, location , capacity, price, level , #artist_id);

WorkshopBooking = (booking_id, booking_date , payment_status , #member_id, #workshop_id);

Follows = (#member_id, #artist_id);

presented_in = (#artwork_id, #exhibition_id);

5. Normalization

The database was normalized up to the Third Normal Form.

First Normal Form (1NF)

All tables contain atomic values. There are no repeating groups or multi-valued attributes. For example, an artwork has one title, one type, and one creation date.

Second Normal Form (2NF)

All non-key attributes depend on the whole primary key. Most tables use a single-column primary key. For associative tables such as Follows and presented_in, the composite key represents the complete relationship, and there are no partial dependencies.

Third Normal Form (3NF)

There are no transitive dependencies between non-key attributes. For example, gallery information is stored in the Gallery table and is referenced by Exhibition using gallery_id, instead of repeating gallery data inside the exhibition table. Similarly, user information is stored once in UserAccount, while role-specific information is separated into CommunityMember, Artist, and Administrator.

SQL Script:

```
CREATE TABLE UserAccount(  
    user_id VARCHAR(50),  
    email VARCHAR(50) NOT NULL,  
    name VARCHAR(50) NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role VARCHAR(50) NOT NULL,  
    PRIMARY KEY(user_id),  
    UNIQUE(email)  
);  
  
CREATE TABLE CommunityMember(  
    member_id VARCHAR(50),  
    join_date DATE NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    membership_type VARCHAR(100) NOT NULL,  
    user_id VARCHAR(50) NOT NULL,  
    PRIMARY KEY(member_id),  
    UNIQUE(user_id),  
    FOREIGN KEY(user_id) REFERENCES UserAccount(user_id)  
);  
  
CREATE TABLE Artist(  
    artist_id VARCHAR(50),  
    biography TEXT NOT NULL,  
    discipline VARCHAR(50) NOT NULL,  
    birth_year INT NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    user_id VARCHAR(50) NOT NULL,  
    PRIMARY KEY(artist_id),  
    UNIQUE(user_id),  
    FOREIGN KEY(user_id) REFERENCES UserAccount(user_id)  
);  
  
CREATE TABLE Administrator(  
    admin_id VARCHAR(50),
```

```

    admin_level VARCHAR(50),
    user_id VARCHAR(50) NOT NULL,
    PRIMARY KEY(admin_id),
    UNIQUE(user_id),
    FOREIGN KEY(user_id) REFERENCES UserAccount(user_id)
);

CREATE TABLE Artwork(
    artwork_id VARCHAR(50),
    title VARCHAR(50) NOT NULL,
    description TEXT NOT NULL,
    type VARCHAR(50) NOT NULL,
    creation_date DATE NOT NULL,
    artist_id VARCHAR(50) NOT NULL,
    PRIMARY KEY(artwork_id),
    UNIQUE(title),
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id)
);

CREATE TABLE Gallery(
    gallery_id VARCHAR(50),
    address VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    rating DOUBLE NOT NULL,
    admin_id VARCHAR(50) NOT NULL,
    PRIMARY KEY(gallery_id),
    FOREIGN KEY(admin_id) REFERENCES Administrator(admin_id)
);

CREATE TABLE Exhibition(
    exhibition_id VARCHAR(50),
    title VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    event_date DATE NOT NULL,
    capacity INT NOT NULL,
    gallery_id VARCHAR(50) NOT NULL,
    PRIMARY KEY(exhibition_id),
    FOREIGN KEY(gallery_id) REFERENCES Gallery(gallery_id)
);

CREATE TABLE Workshop(
    workshop_id VARCHAR(50),
    title VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    workshop_date DATE NOT NULL,
    location VARCHAR(100) NOT NULL,
    capacity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,

```

```

    level VARCHAR(100) NOT NULL,
    artist_id VARCHAR(50) NOT NULL,
    PRIMARY KEY(workshop_id),
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id)
);

CREATE TABLE WorkshopBooking(
    booking_id VARCHAR(50),
    booking_date DATETIME NOT NULL,
    payment_status VARCHAR(100) NOT NULL,
    member_id VARCHAR(50) NOT NULL,
    workshop_id VARCHAR(50) NOT NULL,
    PRIMARY KEY(booking_id),
    FOREIGN KEY(member_id) REFERENCES CommunityMember(member_id),
    FOREIGN KEY(workshop_id) REFERENCES Workshop(workshop_id)
);

CREATE TABLE Follows(
    member_id VARCHAR(50),
    artist_id VARCHAR(50),
    PRIMARY KEY(member_id, artist_id),
    FOREIGN KEY(member_id) REFERENCES CommunityMember(member_id),
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id)
);

CREATE TABLE presented_in(
    artwork_id VARCHAR(50),
    exhibition_id VARCHAR(50),
    PRIMARY KEY(artwork_id, exhibition_id),
    FOREIGN KEY(artwork_id) REFERENCES Artwork(artwork_id),
    FOREIGN KEY(exhibition_id) REFERENCES Exhibition(exhibition_id)
);

```

Step 3 : Database Implementation and Advanced Features

1.Prompt used for sample data generation

Create a sample data insertion script to fill these tables(UserAccount...) that i provided you by respecting the constraints i gave you too. And the result was:

```

INSERT INTO UserAccount(user_id, email, name, password, role) VALUES
('U001','alice@example.com','Alice Smith','password123','COMMUNITY_MEMBER'),
('U002','bob@example.com','Bob Johnson','password123','COMMUNITY_MEMBER'),
('U003','carol@example.com','Carol Williams','password123','COMMUNITY_MEMBER'),
('U004','dave@example.com','Dave Brown','password123','COMMUNITY_MEMBER'),
('U005','emma@example.com','Emma Davis','password123','COMMUNITY_MEMBER'),
('U006','frank@example.com','Frank Wilson','password123','COMMUNITY_MEMBER'),

```



```

('U007','grace@example.com','Grace Lee','password123','COMMUNITY_MEMBER'),
('U008','henry@example.com','Henry Clark','password123','COMMUNITY_MEMBER'),
('U009','isabel@example.com','Isabel Adams','password123','COMMUNITY_MEMBER'),
('U010','jack@example.com','Jack Baker','password123','COMMUNITY_MEMBER'),
('U011','linda@example.com','Linda Carter','password123','COMMUNITY_MEMBER'),
('U012','michael@example.com','Michael Hall','password123','COMMUNITY_MEMBER'),
('U013','nina@example.com','Nina Young','password123','COMMUNITY_MEMBER'),
('U014','oliver@example.com','Oliver King','password123','COMMUNITY_MEMBER'),
('U015','paula@example.com','Paula Scott','password123','COMMUNITY_MEMBER'),
('U016','quentin@example.com','Quentin Wright','password123','COMMUNITY_MEMBER'),
('U017','rachel@example.com','Rachel Green','password123','COMMUNITY_MEMBER'),
('U018','steve@example.com','Steve Hill','password123','ARTIST'),
('U019','tina@example.com','Tina Moore','password123','ARTIST'),
('U020','victor@example.com','Victor Turner','password123','ADMINISTRATOR');

-- -----
-- CommunityMember
-- -----
INSERT INTO CommunityMember(member_id, join_date, city, membership_type, user_id) V
ALUES
('M001','2025-01-15','Paris','BASIC','U001'),
('M002','2025-01-20','Lyon','PREMIUM','U002'),
('M003','2025-01-22','Marseille','BASIC','U003'),
('M004','2025-01-25','Toulouse','BASIC','U004'),
('M005','2025-02-01','Nice','PREMIUM','U005'),
('M006','2025-02-05','Nantes','BASIC','U006'),
('M007','2025-02-07','Strasbourg','PREMIUM','U007'),
('M008','2025-02-10','Bordeaux','BASIC','U008'),
('M009','2025-02-12','Lille','BASIC','U009'),
('M010','2025-02-15','Rennes','PREMIUM','U010'),
('M011','2025-02-18','Paris','BASIC','U011'),
('M012','2025-02-20','Lyon','PREMIUM','U012'),
('M013','2025-02-22','Marseille','BASIC','U013'),
('M014','2025-02-25','Toulouse','BASIC','U014'),
('M015','2025-03-01','Nice','PREMIUM','U015'),
('M016','2025-03-03','Nantes','BASIC','U016'),
('M017','2025-03-05','Strasbourg','PREMIUM','U017'),
('M018','2025-03-07','Bordeaux','BASIC','U018'),
('M019','2025-03-10','Lille','BASIC','U019'),
('M020','2025-03-12','Rennes','PREMIUM','U020');

-- -----
-- Artist
-- -----
INSERT INTO Artist(artist_id, biography, discipline, birth_year, city, user_id) VAL
UES
('A001','Abstract painter using bright colors','PAINTING',1985,'Paris','U018'),
('A002','Classical pianist performing live','MUSIC',1990,'Lyon','U019'),

```

```

('A003','Digital illustrator for games','OTHER',1995,'Marseille','U001'),
('A004','Contemporary sculptor','SCULPTURE',1982,'Toulouse','U002'),
('A005','Urban photographer capturing streets','PHOTOGRAPHY',1993,'Nice','U003'),
('A006','Oil painter for landscapes','PAINTING',1988,'Nantes','U004'),
('A007','Jazz musician','MUSIC',1991,'Strasbourg','U005'),
('A008','Ceramic sculptor','SCULPTURE',1980,'Bordeaux','U006'),
('A009','Watercolor artist','PAINTING',1996,'Lille','U007'),
('A010','Indie singer-songwriter','MUSIC',1994,'Rennes','U008'),
('A011','3D designer for animation','OTHER',1997,'Paris','U009'),
('A012','Installation artist with lights','OTHER',1989,'Lyon','U010'),
('A013','Street muralist','PAINTING',1992,'Marseille','U011'),
('A014','Classical violinist','MUSIC',1987,'Toulouse','U012'),
('A015','Portrait photographer','PHOTOGRAPHY',1995,'Nice','U013'),
('A016','Glass artist','SCULPTURE',1983,'Nantes','U014'),
('A017','Modern abstract painter','PAINTING',1991,'Strasbourg','U015'),
('A018','Electronic music producer','MUSIC',1998,'Bordeaux','U016'),
('A019','Comic artist','OTHER',1996,'Lille','U017'),
('A020','Admin artist sample','OTHER',1984,'Rennes','U020');

```

```

-----
-- Administrator
-----

```

```

INSERT INTO Administrator(admin_id, admin_level, user_id) VALUES
('AD001','SUPER','U020'),
('AD002','STANDARD','U005'),
('AD003','STANDARD','U010'),
('AD004','STANDARD','U015'),
('AD005','STANDARD','U018'),
('AD006','STANDARD','U001'),
('AD007','STANDARD','U002'),
('AD008','STANDARD','U003'),
('AD009','STANDARD','U004'),
('AD010','STANDARD','U006'),
('AD011','STANDARD','U007'),
('AD012','STANDARD','U008'),
('AD013','STANDARD','U009'),
('AD014','STANDARD','U011'),
('AD015','STANDARD','U012'),
('AD016','STANDARD','U013'),
('AD017','STANDARD','U014'),
('AD018','STANDARD','U016'),
('AD019','STANDARD','U017'),
('AD020','STANDARD','U019');

```

```

-----
-- Artwork
-----

```

```

INSERT INTO Artwork(artwork_id, title, description, type, creation_date, artist_id)

```

VALUES

```
('AW001','Sunset Dreams','Abstract painting with warm colors','PAINTING','2024-12-10','A001'),
('AW002','Ocean Whispers','Cool-toned abstract painting','PAINTING','2025-01-05','A001'),
('AW003','Moonlight Sonata','Piano composition in C minor','MUSIC','2025-02-01','A002'),
('AW004','Digital Hero','Character illustration for games','OTHER','2025-01-15','A003'),
('AW005','Urban Shadows','Street photo series','PHOTOGRAPHY','2025-01-20','A005'),
('AW006','Landscape Serenity','Oil painting of mountains','PAINTING','2025-01-25','A006'),
('AW007','Jazz Evening','Live performance recording','MUSIC','2025-02-05','A007'),
('AW008','Clay Vase','Ceramic art','SCULPTURE','2025-02-10','A008'),
('AW009','Botanical Study','Watercolor botanical illustrations','PAINTING','2025-02-12','A009'),
('AW010','Indie Track','Indie music recording','MUSIC','2025-02-15','A010'),
('AW011','3D Hero','3D model design','OTHER','2025-02-18','A011'),
('AW012','Light Installation','Installation with LEDs','OTHER','2025-02-20','A012'),
('AW013','Street Mural','Colorful wall mural','PAINTING','2025-02-22','A013'),
('AW014','Violin Sonata','Classical violin piece','MUSIC','2025-02-25','A014'),
('AW015','Portrait Shot','Studio portrait photography','PHOTOGRAPHY','2025-03-01','A015'),
('AW016','Glass Sculpture','Decorative glass','SCULPTURE','2025-03-03','A016'),
('AW017','Modern Abstract','Experimental abstract painting','PAINTING','2025-03-05','A017'),
('AW018','Electronic Mix','Electronic music track','MUSIC','2025-03-07','A018'),
('AW019','Comic Panel','Illustrated comic page','OTHER','2025-03-10','A019'),
('AW020','Admin Sample Art','Sample artwork for admin','OTHER','2025-03-12','A020');
```

-- -----

-- Gallery

-- -----

INSERT INTO Gallery(gallery_id, address, name, rating, admin_id) VALUES

```
('G001','12 Rue de Rivoli, Paris','Louvre Art House',4.9,'AD001'),
('G002','25 Avenue Victor Hugo, Lyon','Modern Vision Gallery',4.7,'AD002'),
('G003','8 Rue Nationale, Marseille','Mediterranean Art Space',4.6,'AD003'),
('G004','15 Place du Capitole, Toulouse','Southern Gallery',4.5,'AD004'),
('G005','4 Promenade des Anglais, Nice','Blue Coast Gallery',4.8,'AD005'),
('G006','10 Rue Cr billon, Nantes','Creative Hub',4.4,'AD006'),
('G007','18 Grand Rue, Strasbourg','Alsace Art Center',4.7,'AD007'),
('G008','9 Quai des Chartrons, Bordeaux','River Art Gallery',4.6,'AD008'),
('G009','3 Rue Faidherbe, Lille','Northern Art Space',4.5,'AD009'),
('G010','6 Rue Saint-Georges, Rennes','Rennes Art House',4.3,'AD010');
```

-- -----

```

-- Exhibition
-- -----
INSERT INTO Exhibition(exhibition_id, title, description, event_date, capacity, gal
lery_id) VALUES
('EX001','Abstract Art Exhibition','Showcasing abstract paintings','2025-03-15',5
0,'G001'),
('EX002','Watercolor Exhibition','Botanical paintings display','2025-04-20',40,'G00
2'),
('EX003','3D Art Exhibition','3D digital designs','2025-04-25',30,'G003'),
('EX004','Light Installation Show','LED installation display','2025-04-27',35,'G00
4'),
('EX005','Photography Exhibit','Portrait photos on display','2025-05-05',25,'G00
5'),
('EX006','Glass Art Exhibit','Decorative glass display','2025-05-08',20,'G006'),
('EX007','Modern Abstract Show','Experimental paintings exhibition','2025-05-10',5
0,'G007'),
('EX008','Comic Con Display','Comic panels and artworks','2025-05-15',100,'G008'),
('EX009','Spring Art Fair','Various artworks exhibition','2025-05-20',150,'G009'),
('EX010','Urban Shadows Expo','Street photography exhibition','2025-05-25',60,'G01
0');

-- -----
-- Workshop
-- -----
INSERT INTO Workshop(workshop_id, title, description, workshop_date, location, capa
city, price, level, artist_id) VALUES
('W001','Digital Art Workshop','Character drawing workshop','2025-04-05','Community
Center',30,49.99,'BEGINNER','A003'),
('W002','Landscape Painting Class','Oil painting lesson','2025-04-12','Studio One',
25,59.99,'INTERMEDIATE','A006'),
('W003','Ceramic Workshop','Hands-on pottery workshop','2025-04-18','Art Studio',1
5,69.99,'BEGINNER','A008'),
('W004','Street Photography Tour','Urban photography event','2025-04-10','City Stre
ets',20,39.99,'BEGINNER','A005'),
('W005','Jazz Improvisation','Learn jazz improvisation basics','2025-04-15','Jazz C
lub',20,79.99,'ADVANCED','A007'),
('W006','Watercolor Basics','Introduction to watercolor painting','2025-04-22','Gal
lery Two',30,44.99,'BEGINNER','A009'),
('W007','3D Modeling Basics','Introduction to 3D design','2025-04-28','Digital La
b',18,89.99,'INTERMEDIATE','A011'),
('W008','Violin Masterclass','Classical violin training session','2025-05-02','Conc
ert Hall',12,99.99,'ADVANCED','A014'),
('W009','Comic Drawing Workshop','Create your first comic panel','2025-05-15','Conv
ention Hall',25,54.99,'BEGINNER','A019'),
('W010','Electronic Music Production','Create electronic music tracks','2025-05-1
2','Club One',20,74.99,'INTERMEDIATE','A018');

-- -----

```

```

-- WorkshopBooking
-----
INSERT INTO WorkshopBooking(booking_id, booking_date, payment_status, member_id, workshop_id) VALUES
('B001', '2025-03-01 10:00:00', 'PAID', 'M001', 'W001'),
('B002', '2025-03-05 11:30:00', 'PAID', 'M001', 'W002'),
('B003', '2025-03-08 09:15:00', 'PENDING', 'M002', 'W001'),
('B004', '2025-03-10 14:00:00', 'PAID', 'M003', 'W003'),
('B005', '2025-03-12 16:45:00', 'PAID', 'M004', 'W004'),
('B006', '2025-03-15 12:00:00', 'PENDING', 'M005', 'W005'),
('B007', '2025-03-17 13:30:00', 'PAID', 'M006', 'W006'),
('B008', '2025-03-20 15:00:00', 'PAID', 'M007', 'W007'),
('B009', '2025-03-22 17:20:00', 'PAID', 'M008', 'W008'),
('B010', '2025-03-25 10:10:00', 'PAID', 'M009', 'W009'),
('B011', '2025-03-27 11:50:00', 'PAID', 'M010', 'W010'),
('B012', '2025-03-30 09:00:00', 'PENDING', 'M011', 'W001'),
('B013', '2025-04-02 10:30:00', 'PAID', 'M012', 'W002'),
('B014', '2025-04-05 14:15:00', 'PAID', 'M013', 'W003'),
('B015', '2025-04-07 16:00:00', 'PAID', 'M014', 'W004'),
('B016', '2025-04-10 13:45:00', 'PAID', 'M015', 'W005'),
('B017', '2025-04-12 15:20:00', 'PAID', 'M016', 'W006'),
('B018', '2025-04-15 18:00:00', 'PENDING', 'M017', 'W007'),
('B019', '2025-04-18 12:40:00', 'PAID', 'M018', 'W008'),
('B020', '2025-04-20 10:25:00', 'PAID', 'M019', 'W009');

-----
-- Follows
-----
INSERT INTO Follows(member_id, artist_id) VALUES
('M001', 'A001'),
('M001', 'A002'),
('M002', 'A001'),
('M002', 'A003'),
('M003', 'A002'),
('M003', 'A004'),
('M004', 'A005'),
('M004', 'A006'),
('M005', 'A003'),
('M005', 'A007'),
('M006', 'A004'),
('M006', 'A008'),
('M007', 'A009'),
('M007', 'A010'),
('M008', 'A001'),
('M008', 'A002'),
('M009', 'A003'),
('M009', 'A004'),
('M010', 'A005'),

```

```

('M010', 'A006');

-- -----
-- presented_in
-- -----

INSERT INTO presented_in(artwork_id, exhibition_id) VALUES
('AW001', 'EX001'),
('AW002', 'EX001'),
('AW009', 'EX002'),
('AW011', 'EX003'),
('AW012', 'EX004'),
('AW015', 'EX005'),
('AW016', 'EX006'),
('AW017', 'EX007'),
('AW019', 'EX008'),
('AW001', 'EX009'),
('AW003', 'EX009'),
('AW005', 'EX010'),
('AW006', 'EX009'),
('AW008', 'EX006'),
('AW013', 'EX007');

```

2. Views

View	Objective
vw_artist_summary	This view combines the Artist table with the UserAccount table to display a simplified artist profile. It shows the artist ID, name, discipline, birth year, and city. It is useful because the artist's name is stored in UserAccount, while artist-specific details are stored in Artist.
vw_exhibition_summary	This view combines Exhibition with Gallery. It shows each exhibition with its title, date, capacity, and gallery name. It simplifies queries because the application can display exhibition information without manually joining the two tables each time.
vw_users_sanitized	This view displays basic user information: user ID, name, email, and role. It excludes the password field, which makes it safer to use when displaying user information in the application or reports.
vw_member_workshop_bookings	This view combines CommunityMember, UserAccount, WorkshopBooking, and Workshop. It shows which member booked which workshop, along with the booking date and payment status. It is useful for booking history and administration screens.
vw_artist_followers	This view combines Follows, Artist, CommunityMember, and UserAccount. It shows which members follow each artist. It helps analyze artist popularity and community engagement.
vw_artworks_in_exhibitions	This view combines presented_in, Artwork, and Exhibition. It shows which artworks are presented in which exhibitions, including the exhibition date. It simplifies the many-to-many relationship between artworks and exhibitions.

3. Indexes

Index	Table	Objective
idx_artist_discipline	Artist	Speeds up filtering artists by discipline
idx_artist_city	Artist	Speeds up filtering artists by city
idx_artwork_artist	Artwork	Improves joins between artworks and artists
idx_gallery_admin	Gallery	Improves joins between galleries and administrators
idx_exhibition_date	Exhibition	Speeds up search by exhibition date
idx_exhibition_gallery	Exhibition	Improves joins between exhibitions and galleries
idx_workshop_artist	Workshop	Improves joins between workshops and artists
idx_workshop_date	Workshop	Speeds up filtering workshops by date
idx_booking_workshop_member	WorkshopBooking	Optimizes booking lookups by workshop and member
idx_follow_member_artist	Follows	Optimizes follow relationship lookups
idx_presented_exhibition	presented_in	Optimizes exhibition-artwork lookups

4. Triggers

Trigger	Objective
trg_check_workshop_capacity	This trigger runs before inserting a new workshop. It checks that the workshop capacity is greater than zero. If the capacity is invalid, the database rejects the insertion and returns an error message.
trg_workshop_capacity_check	This trigger runs before inserting a new workshop booking. It retrieves the current capacity of the selected workshop. If the capacity is zero or negative, it prevents the booking and displays the message "Workshop is full".
trg_update_workshop_capacity	This trigger runs after inserting a new workshop booking. It automatically decreases the capacity of the booked workshop by one. This keeps the available number of places updated without requiring the application to do it manually.
trg_prevent_duplicate_follow	This trigger runs before inserting a new follow relationship. It checks whether the same member already follows the same artist. If the relationship already exists, the insertion is rejected to avoid duplicate follows.

5. Stored Procedures and Function

Stored program	Type	Objective
sp_create_workshop	Procedure	Creates a workshop after checking that capacity is positive
sp_book_workshop	Procedure	Books a workshop and prevents duplicate booking or full capacity
sp_follow_artist	Procedure	Allows a member to follow an artist if not already following

Stored program	Type	Objective
sp_cancel_workshop_booking	Procedure	Cancels a booking and restores workshop capacity
fn_workshop_booking_count	Function	Returns the number of paid bookings for a workshop

6.Transaction test script

A transaction was used to test an atomic operation where one member books multiple workshops. All bookings must succeed together, or none of them should be saved. This transaction makes sure that the group of booking operations is handled as one unit. If an error occurs during one of the bookings, the transaction can be rolled back to avoid partial changes.

```
START TRANSACTION;

CALL sp_book_workshop('B021', 'M020', 'W001');
CALL sp_book_workshop('B022', 'M020', 'W002');
CALL sp_book_workshop('B023', 'M020', 'W003');

COMMIT;
```

Step 4: Integrating the Database into the ArtConnect Java Application

1. DAO Layer

The project uses the DAO pattern, which separates database access from the rest of the application logic. DAO interfaces are located in: `com.project.artconnect.dao`

DAO interface	Role
ArtistDao	Defines operations for artists
ArtworkDao	Defines operations for artworks
CommunityMemberDao	Defines operations for community members
ExhibitionDao	Defines operations for exhibitions
GalleryDao	Defines operations for galleries
WorkshopDao	Defines operations for workshops

Each DAO defines common CRUD operations such as: `findAll()`, `save()`, `update()`, `delete()` and some of them also define search methods.

The JDBC implementations are located in: `com.project.artconnect.persistence`

JDBC class	Implements
JdbcArtistDao	ArtistDao
JdbcArtworkDao	ArtworkDao
JdbcCommunityMemberDao	CommunityMemberDao
JdbcExhibitionDao	ExhibitionDao

JDBC class	Implements
JdbcGalleryDao	GalleryDao
JdbcWorkshopDao	WorkshopDao

These classes use JDBC to communicate with the MySQL database. SQL queries are executed using PreparedStatement, which is safer and more reliable than directly building SQL strings. Database resources are handled using try-with-resources, allowing connections, statements, and result sets to be closed automatically.

2. Database Connection

The database connection is configured in the DatabaseConfig class:

```
public class DatabaseConfig {
    public static final String URL = "jdbc:mysql://localhost:3306/ArtConnect";
    public static final String USER = "root";
    public static final String PASSWORD = "root1234";
}
```

The connection is created by the ConnectionManager utility class:

```
public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(
        DatabaseConfig.URL,
        DatabaseConfig.USER,
        DatabaseConfig.PASSWORD
    );
}
```

This centralizes the database connection logic. Instead of writing the connection code in every DAO, all JDBC classes call: ConnectionManager.getConnection()

3. Service Layer Adaptation

The service layer is located in: com.project.artconnect.service and its JDBC implementations are located in: com.project.artconnect.service.impl

The service interfaces define the operations used by the application, while the JDBC service implementations call the DAO layer.

Service implementation	Uses
JdbcArtistService	ArtistDao
JdbcArtworkService	ArtworkDao
JdbcCommunityService	CommunityMemberDao
JdbcExhibitionService	ExhibitionDao
JdbcGalleryService	GalleryDao
JdbcWorkshopService	WorkshopDao

For example, JdbcArtworkService does not directly write SQL queries. Instead, it calls ArtworkDao, and the DAO is responsible for database access.

4. Service Provider

The application uses a ServiceProvider class to create and provide the correct service objects.java

```
private static final ArtistDao artistDao = new JdbcArtistDao();
private static final ArtworkDao artworkDao = new JdbcArtworkDao();
private static final GalleryDao galleryDao = new JdbcGalleryDao();
private static final WorkshopDao workshopDao = new JdbcWorkshopDao();
private static final CommunityMemberDao communityMemberDao = new JdbcCommunityMemberDao();
private static final JdbcExhibitionDao exhibitionDao = new JdbcExhibitionDao();
```

Then, each DAO is injected into its corresponding service:

```
private static final ArtworkService artworkService =
    new JdbcArtworkService(artworkDao);
```

5. UI Adaptation

The UI controllers are located in:com.project.artconnect.ui

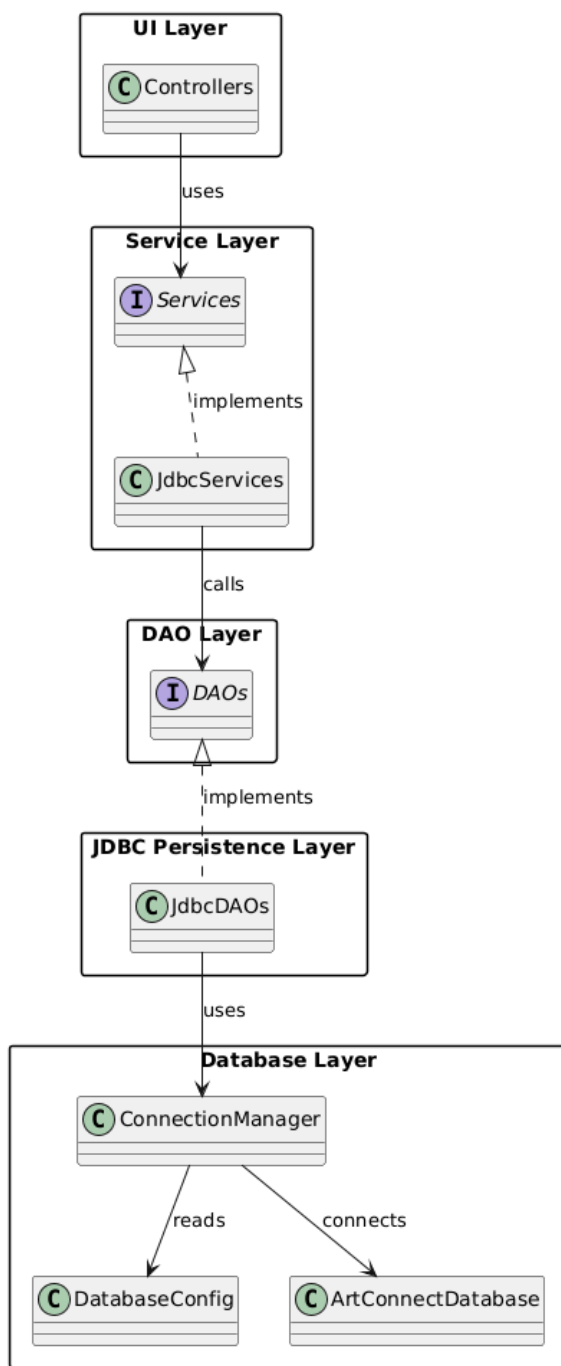
The main controllers are:

Controller	Purpose
DiscoverController	Displays featured exhibitions and workshops
ArtistController	Manages artists
ArtworkController	Manages artworks
GalleryController	Manages galleries
ExhibitionController	Manages exhibitions
WorkshopController	Manages workshops
CommunityController	Manages community members

The controllers access the service layer through ServiceProvider.

6. Application Architecture

ArtConnect Layered Architecture



Layer	Responsibility
UI layer	Displays data and handles user actions
Service layer	Contains application logic
DAO layer	Defines database operations
JDBC persistence layer	Executes SQL queries
Database layer	Stores persistent data

Screenshots of the ArtConnect + the Database

File Help

Discover Artists Artworks Galleries Exhibitions Workshops Community

Search artists...

Filter by Discipline Search Reset

Name: bezhkq Email: ej@gmail.com

City: paris Birth year: 1999

Biography: fsjn Discipline: SCULPTURE

Add Artist Update Artist Delete Artist

Discover Artists Artworks Galleries Exhibitions Workshops Community

Search artists...

Filter by Discipline Search Reset

Name: Artist name Email: Email

City: City Birth year: Birth year

Biography: Biography Discipline:

Add Artist Update Artist Delete Artist

Success
Artist added successfully.
OK

Name	City	Email	Birth Year	Discipline	Biography
Alice Smith	Marseille	alice@example.com	1995	OTHER	Digital illustrator for games
bezhkq	paris	ej@gmail.com	1999	SCULPTURE	fsjn
Bob Johnson	Toulouse	bob@example.com	1982	SCULPTURE	Contemporary sculptor
Carol Williams	Nice	carol@example.com	1993	PHOTOGRAPHY	Urban photographer capturing streets
Dave Brown	Nantes	dave@example.com	1988	PAINTING	Oil painter for landscapes
Emma Davis	Strasbourg	emma@example.com	1991	MUSIC	Jazz musician
Frank Wilson	Bordeaux	frank@example.com	1980	SCULPTURE	Ceramic sculptor
Grace Lee	Lille	grace@example.com	1996	PAINTING	Watercolor artist
Henry Clark	Rennes	henry@example.com	1994	MUSIC	Indie singer-songwriter
Isabel Adams	Paris	isabel@example.com	1997	OTHER	3D designer for animation
Jack Baker	Lyon	jack@example.com	1989	OTHER	Installation artist with lights
Linda Carter	Marseille	linda@example.com	1992	PAINTING	Street muralist
Michael Hall	Toulouse	michael@example.com	1987	MUSIC	Classical violinist

It was added in the second row:

Name	City	Email	Birth Year	Discipline	Biography
Alice Smith	Marseille	alice@example.com	1995	OTHER	Digital illustrator for games
bezhkq	paris	ej@gmail.com	1999	SCULPTURE	fsjn
Bob Johnson	Toulouse	bob@example.com	1982	SCULPTURE	Contemporary sculptor
Carol Williams	Nice	carol@example.com	1993	PHOTOGRAPHY	Urban photographer capturing streets
Dave Brown	Nantes	dave@example.com	1988	PAINTING	Oil painter for landscapes

and in the Database it was stocked in UserAccount (last one):

user_id	email	name	password	role	
U003	carol@example.com	Carol Williams	password123	COMMUNITY_MEMBER	
U004	dave@example.com	Dave Brown	password123	COMMUNITY_MEMBER	
U005	emma@example.com	Emma Davis	password123	COMMUNITY_MEMBER	
U006	frank@example.com	Frank Wilson	password123	COMMUNITY_MEMBER	
U007	grace@example.com	Grace Lee	password123	COMMUNITY_MEMBER	
U008	henry@example.com	Henry Clark	password123	COMMUNITY_MEMBER	
U009	isabel@example.com	Isabel Adams	password123	COMMUNITY_MEMBER	
U010	jack@example.com	Jack Baker	password123	COMMUNITY_MEMBER	
U011	linda@example.com	Linda Carter	password123	COMMUNITY_MEMBER	
U012	michael@example.c...	Michael Hall	password123	COMMUNITY_MEMBER	
U013	nina@example.com	Nina Young	password123	COMMUNITY_MEMBER	
U014	oliver@example.com	Oliver King	password123	COMMUNITY_MEMBER	
U015	paula@example.com	Paula Scott	password123	COMMUNITY_MEMBER	
U016	quentin@example.com	Quentin Wright	password123	COMMUNITY_MEMBER	
U017	rachel@example.com	Rachel Green	password123	COMMUNITY_MEMBER	
U018	steve@example.com	Steve Hill	password123	ARTIST	
U019	tina@example.com	Tina Moore	password123	ARTIST	
U020	victor@example.com	Victor Turner	password123	ADMINISTRATOR	
U78A...	ej@gmail.com	bezhkq	defaultPass...	ARTIST	
NULL	NULL	NULL	NULL	NULL	

and in Artist Table we have it's informations(last one):

artist_id	biography	discipline	birth_year	city	user_id	
A003	Digital illustrator for games	OTHER	1995	Marseille	U001	
A004	Contemporary sculptor	SCULPTURE	1982	Toulouse	U002	
A005	Urban photographer capturing streets	PHOTOGRAPHY	1993	Nice	U003	
A006	Oil painter for landscapes	PAINTING	1988	Nantes	U004	
A007	Jazz musician	MUSIC	1991	Strasbourg	U005	
A008	Ceramic sculptor	SCULPTURE	1980	Bordeaux	U006	
A009	Watercolor artist	PAINTING	1996	Lille	U007	
A010	Indie singer-songwriter	MUSIC	1994	Rennes	U008	
A011	3D designer for animation	OTHER	1997	Paris	U009	
A012	Installation artist with lights	OTHER	1989	Lyon	U010	
A013	Street muralist	PAINTING	1992	Marseille	U011	
A014	Classical violinist	MUSIC	1987	Toulouse	U012	
A015	Portrait photographer	PHOTOGRAPHY	1995	Nice	U013	
A016	Glass artist	SCULPTURE	1983	Nantes	U014	
A017	Modern abstract painter	PAINTING	1991	Strasbourg	U015	
A018	Electronic music producer	MUSIC	1998	Bordeaux	U016	
A019	Comic artist	OTHER	1996	Lille	U017	
A020	Admin artist sample	OTHER	1984	Rennes	U020	
A79D...	fsjn	SCULPTURE	1999	paris	U78A...	
NULL	NULL	NULL	NULL	NULL	NULL	

Step 5: Finalization and Presentation

1. Refinement and Testing

After integrating the database into the Java application, several tests were performed to verify the consistency between the application and the database.

The first part of the testing focused on checking that the database schema matched the needs of the Java application. The main entities used in the interface, such as artists, artworks, galleries, exhibitions, workshops, and community members, were connected to their corresponding database tables.

The application was tested to make sure that the main CRUD operations worked correctly:

Scenario	Expected result
Display artists	Artists are loaded from the Artist and UserAccount tables
Add a new artist	The artist is inserted into the database

Scenario	Expected result
Update an artist	The artist information is modified and saved
Delete an artist	The artist is removed from the database
Display artworks	Artworks are loaded from the Artwork table
Add or update an artwork	The changes are saved in the database
Display galleries	Galleries are loaded from the Gallery table
Display exhibitions	Exhibitions are loaded with their gallery information
Display workshops	Workshops are loaded with artist and workshop details
Display community members	Members are loaded from the CommunityMember and UserAccount tables

The database constraints were also checked. For example, capacities must be positive, payment status must use accepted values, and artwork types must match the allowed categories. These constraints help prevent invalid data from being inserted.

Triggers and stored procedures were tested to verify automatic behavior. For example, when a workshop booking is inserted, the workshop capacity is automatically reduced. Duplicate follow relationships are also prevented by the database.